

Passive Pre-Association Wi-Fi Access-Point Selection

U.K.Arvindan
EP21B006

Abstract— A Wi-Fi client in overlapping coverage must select an access point (AP) before it can directly measure the service that each AP would provide. Signal strength describes link quality, but not contention or load, and may therefore favor a nearby but congested AP. This project investigates whether a client can make a better decision using only passive observations collected before association, without probe requests or assistance from the network. This report defines the experimental foundation of that investigation: a controlled ns-3 simulator that creates multi-AP deployments, records decoded frames and physical carrier-sense activity, and replays each fixed topology once per target AP, changing only the AP the candidate joins. Every replay of a topology therefore shares one identical pre-association observation while yielding its own measured throughput. The simulator separates physical-topology randomness from radio randomness and produces a structured choice-set dataset. Across 1500 choice sets, a gradient-boosted model over 35 passive features selects the best AP in 93.4% of cases, against 64.7% for strongest signal strength and 86.7% for a heuristic that also accounts for channel occupancy. Sequence models over the same observation do not improve on this, and the report sets out what a further iteration would need for the value of temporal structure to become testable.

I. PROBLEM FORMULATION

Throughput depends on channel contention, competing stations, transmission rates, retries, and the airtime consumed by neighboring basic service sets. Received signal strength reflects such effects poorly if at all. Those effects do, however, leave traces a client can hear without transmitting: the frames themselves, and the time the medium spends busy. This work asks whether such traces are enough: *can a client choose the AP that will provide the greatest throughput using only what its radio can passively observe before association?*

Existing load-aware methods [1], [2] make their load estimate after association, from AP-provided reports, or from a network controller. A client making its first choice has none of these: only beacons and some data frames, in which AP distance and offered traffic appear only through their effects.

Let $\mathcal{A} = \{a_1, \dots, a_N\}$ be the APs in one deployment and let O be the candidate station's passive observation before it joins any AP. If y_i is the throughput obtained after joining a_i , the desired decision is

$$a^* = \arg \max_{a_i \in \mathcal{A}} y_i, \quad (1)$$

using a decision function whose inputs are derived only from O . In a live network only one y_i is ever measurable: the station joins a single AP and observes that AP's throughput, while the values of y_j for every AP it did not join are counterfactual. The simulator recovers all N values by replaying the same scenario once per target AP, changing only the AP the candidate joins. Every choice set therefore carries its true a^* .

The replay is sound only if O is identical across all N runs of a scenario. If any part of the observation varied with the AP eventually joined, a decision function could reach the right answer through a simulation artifact rather than through the network. This is checked rather than assumed. The assumptions that do remain are stated next.

II. EXPERIMENTAL SCOPE AND ASSUMPTIONS

The simulator isolates the relationship among path quality, background load, and passive evidence. It does not attempt to reproduce every feature of a deployed WLAN. Table I states the principal assumptions so that conclusions can be interpreted within the intended scope.

TABLE I
CONTROLLED ASSUMPTIONS OF THE SIMULATOR.

Aspect	Current assumption
Deployment	Static, single-floor, no walls; one stationary candidate and stationary background stations.
Radios	Homogeneous IEEE 802.11n devices at 5 GHz with 20 MHz channels and Minstrel-HT rate control [5].
Propagation	Yans Wi-Fi channel; log-distance loss (exponent 3.0, 46.6777 dB reference loss at 1 m) followed by Nakagami fading.
Traffic	Uplink UDP; constant background offered rates and a saturated candidate. Wired backhaul is not a bottleneck.
Observation	Passive and client-side; no probe request, association, or network-provided load report during feature collection.
Target	Mean candidate uplink throughput from successful association until the simulation stops; failed association is assigned zero.

The choice set presented to the candidate station contains only APs discovered during its passive scan.

The environment is static and open, so it excludes mobility, walls, hardware diversity, and application demand that varies over time. The label is uplink UDP throughput, so it says little about TCP goodput, downlink performance, or latency.

III. SIMULATOR DESIGN

The simulator is built on ns-3.45 and its Wi-Fi models [5]. APs sit 30 m apart in a fixed layout: two form a pair, three an

equilateral triangle, and four, six, and eight fill 2×2 , 3×2 , and 4×2 grids. Channels 36, 40, 44, and 48 are assigned in AP-index order and reused once a deployment exceeds four APs. Because the layout is fixed as well, co-channel separation takes a single value per deployment size — 30 m at eight APs, 42 m at six, and no reuse at or below four — which ties channel reuse to AP count. Section VI returns to this.

Each scenario draws 2, 3, 4, 6, or 8 APs and 2, 3, or 4 background stations per AP. A quarter of scenarios spread those stations uniformly. The rest place one or more hotspot APs, with fewer hotspots more likely and the count capped at $\lceil N_{AP}/3 \rceil$. Around those hotspots 70% of background stations fall in 10 m discs, and the remainder cover the footprint. Mixing uniform and deliberately clustered regimes follows common practice in WLAN studies [3], [4], and matters here because under uniform load every AP is about equally busy and the choice collapses back to signal strength. Each station associates with its nearest AP, and every background station in a scenario offers the same load, drawn uniformly at random from 1.0, 2.5, 4.0, and 6.0 Mbit/s. Candidate positions are drawn to keep the decision non-trivial: 70% near a boundary between neighboring APs, 30% at 4–16 m from a single AP. These are sampling policies rather than radio parameters, and they are recorded with the dataset because they set its difficulty.

Background traffic starts at $t = 1$ s. Passive recording runs until $t = 5.5$ s, covering 4.5 s of loaded traffic; the candidate associates at $t = 6$ s and the run ends at $t = 18$ s. The 0.5 s gap between recording and association is a guard: ns-3 performs target-dependent polling just before a join, and without the gap that polling would land inside the observation window. After joining, the candidate offers 100 Mbit/s in 1250-byte packets, which keeps it backlogged so that measured throughput reflects the service the network gives it rather than the limits of its own traffic generator.

A. Matched Counterfactual Replays

For each sampled topology and radio seed, the experiment runs once per target AP. AP and station positions, background traffic, the candidate’s position, and every pre-association device stay fixed; only the AP selected at $t = 6$ s changes. A `topologySeed` controls background-station placement and an independent `rngSeed` controls fading, contention, and rate adaptation, with five radio seeds generated per topology. Separating the two streams is what allows stochastic radio behavior to be repeated without the physical deployment shifting underneath it.

B. Validation

Validation runs before any model is fitted, so that a good accuracy number cannot mask a defective data-generating process.

Three basic checks ensure that the above outlined design is actualised automatically: (i) Repeating a seed reproduces throughput exactly (any run can be regenerated), (ii) Changing only the radio seed varies the outcome while every physical

position stays fixed, and (iii) The recorded frame and channel-busy traces are byte-identical across all target-AP replays of a matched scenario (this enforces the constant O required in Section I).

These establish internal consistency, not that the simulator behaves like a real network.

IV. OBSERVATION AND DATASET CONSTRUCTION

A. Recording and Projection

The simulator separates data collection from what a deployable client is assumed to observe. During simulation, receive-only scanner PHYs record activity on every occupied channel simultaneously. This produces a complete passive record that is identical across the matched target-AP replays. After simulation, that record is projected onto a single-radio scan schedule in order to train models under a realistic condition.

Specifically, the candidate carries one receive-only scanner PHY per occupied channel, plus a separate PHY reserved for association. The scanner PHYs never transmit and never associate; the association PHY is parked on unused channel 149 until the decision time, so the AP the candidate will eventually join cannot alter what is heard, or consume random draws, while the observation is being collected. At $t = 5.5$ s the scanner PHYs move to the parking channel as well.

Watching every channel at once is not a claim that real clients have several radios. It is a recording device: it captures a superset of what any single radio could have heard, from which a single-radio scan projection is cut afterwards.

Two traces are recorded. The monitor-sniffer trace holds every frame a scanner decodes, with time, frequency, BSSID, transmitter, frame category, beacon and retry flags, length, RSSI, noise, airtime, and PHY rate. The PHY-state trace gives the fraction of each 102.4 ms beacon interval in which carrier sense marked the channel busy. This clear-channel-assessment (CCA) measurement registers energy a frame-only view misses: interference, and transmissions too distant to decode. Some driver stacks expose an equivalent channel-busy counter, though availability varies by platform.

The superset is then projected onto one passive radio. That radio rotates through the occupied channels in random order, dwelling 110 ms on each with 0.15 ms to retune, and repeats the cycle for as much of the window as it fits, so every channel is revisited many times before the decision. Frames and CCA intervals falling outside a dwell are discarded. A 110 ms dwell spans just over one 102.4 ms beacon interval, so each visit yields about one beacon from every AP on that channel. RSSI is then perturbed by uniform error within ± 3 dB and quantized to 1 dB. Both steps run after ns-3 from a recorded scan seed, so scan policy and measurement noise can be varied without repeating a simulation.

B. From Trace to Training Data

The natural representation is the trace itself: every decoded frame in order, carrying its arrival time, channel, BSSID and transmitter, class, flags, length, signal, airtime, and PHY rate. It is a time series, and we train a transformer on it directly.

The raw observation is a long sequence, and what distinguishes one AP from another does not reside at frame granularity. Both problems that arise here are resolved by aggregating, which is possible only for quantities that can be averaged over a set of frames. We define a list of hand crafted features for this purpose; they are enumerated in Appendix A.

Aggregation is done per AP before the data is bucketed. Frames are attributed to an AP by BSSID. Time is bucketed by channel visit: one bin per dwell, so that no bin spans a retune and aggregates traffic from two channels into a figure representing neither. A 5.5 s window against a 110 ms dwell gives roughly 50 bins. Each bin carries two sets of quantities: one aggregated over that AP’s own frames, the other over all traffic the radio heard during the visit. The second is shared by every AP in the choice set, since one radio hears one channel at a time; a flag records whether the channel being visited was that AP’s own. A transformer attends over bins exactly as it does over frames.

Widening the bucket to the whole window removes time altogether and collapses everything into a single row of tabular data per AP (Table II). AP-specific quantities are aggregated over that AP’s frames alone; channel-specific quantities over the channel that AP occupies. Gradient boosting, random forests, and ridge regression are trained on this representation.

TABLE II
WHAT EACH MODEL RECEIVES FOR ONE CHOICE SET OF N OPTIONS.

Form	Per option	Consumed by
Frames	the decoded trace, one token per frame	Transformer
Binned	~ 50 intervals $\times$ 29 measurements	Transformer
Vector	35 measurements	GBM, RF, ridge

Squashing the time axis in this way carries no comparison between options, since it aggregates over one AP alone: the row records how busy an AP is, not whether it is busier than its neighbors. A set of relative features is therefore engineered and appended, restoring for the tabular models the comparison that per-AP aggregation removed. A sequence or set model needs no such repair, because its set stage attends across options directly.

The five radio-seed realizations of a topology are near-duplicates, so train/test splits are cut by topology: letting two of them fall on opposite sides would inflate every result reported below.

V. MODELS, PROTOCOL, AND RESULTS

The dataset holds 6449 AP options across 1500 choice sets drawn from 300 topologies, a median of 4 options per decision. The best option in a set yields 42.6 Mbit/s on average against 22.0 for one chosen at random, and the best and worst differ by 35.5 Mbit/s.

Every decision function below reads the same observation, and they differ only in how much of it they are handed and in what shape. The comparison is therefore between representations as much as between architectures.

A. Baselines

Three passive baselines are evaluated alongside random selection:

- *Strongest RSSI* ranks options by mean beacon level.
- *Least busy channel* ranks them by the clear-channel-assessment busy fraction of the channel each occupies, so options sharing a channel are indistinguishable to it; tied options are scored at their mean throughput.
- *RSSI minus busy* maps RSSI from -100 to -60 dBm onto the unit interval and subtracts the busy fraction from it.

B. Learned Models

Ridge regression, random forests, gradient boosting, a pointwise MLP and a set ranker take the single-row vector. Transformers take the binned sequence and the frame trace. One further pair of transformer runs takes the vector and the sequence together, which is the direct test of what the trajectory adds to the averages.

1) *Tabular Models*: Tabular models take the 35-feature vector and three encodings of the binned sequence: its average over time, a set of summary statistics per measurement, and the sequence flattened with bin order preserved. Gradient boosting and a pointwise MLP take all three encodings; ridge takes the flattened one only. Both neural models are single hidden-layer networks; the set ranker scores an option against the rest of its choice set, the pointwise MLP scores each row alone.

Every model on the vector is selected per split by validation regret: ridge over regularization strength, gradient boosting over learning rate (0.03 to 0.1), leaf size (4 to 16) and 200 or 400 iterations, the random forest over leaf size and feature fraction at 400 trees, and the neural models over weight decay and score temperature. Both tree models also choose whether to fit a log-transformed target. On the sequence encodings gradient boosting is tuned the same way; the MLPs there use one fixed setting.

2) *Transformers*: Both transformers use a model dimension of 48, four attention heads, and dropout 0.1, and hold roughly 10^5 parameters. The binned model encodes each option’s sequence with two self-attention layers over bins, reads out a learned summary token, then compares options with two further layers over the option set, masked so padding cannot contribute. The frame-level model cannot attend over the raw sequence directly, since a trace runs to thousands of frames; four learned queries per option cross-attend into it instead, which is linear rather than quadratic in frame count. Both are trained with AdamW at 2×10^{-3} , weight decay 10^{-3} , gradient clipping at unit norm, and early stopping on validation regret.

C. Protocol

The learned models divide between two objectives. Regression predicts $\log(1 + \text{throughput})$ in Mbit/s, so its outputs carry a scale. Ranking learns a distribution over the options in a choice set and optimizes the decision directly, at the cost of scores that carry no scale. Only the transformers are run under both.

Each result averages five 60/20/20 partitions of the topologies, stratified by AP count and redrawn per seed, so test sets overlap and the spread is draw-to-draw variation rather than a cross-validation error. Every model sees the same partitions; the frame-level model was run on the first three. Neural models keep one initialization throughout, so their spread excludes initialization variance. Model selection uses regret against the oracle choice on the validation split; the reported metric is top-1 accuracy with a 0.5 Mbit/s tie tolerance.

TABLE III
TOP-1 ACCURACY, MEAN AND STANDARD DEVIATION OVER FIVE PARTITIONS, EXCEPT †, OVER THREE. HEURISTICS ARE SCORED ON THE FEATURE-VECTOR PIPELINE.

Model	Obj.	Top-1
<i>Heuristics</i>		
Random choice	—	0.282 ± 0.002
Least busy channel	—	0.442 ± 0.072
Strongest RSSI	—	0.647 ± 0.042
RSSI minus busy	—	0.867 ± 0.038
<i>Vector: 35 hand-built features</i>		
Ridge	reg.	0.881 ± 0.045
MLP, pointwise	rank	0.897 ± 0.024
Set ranker	rank	0.909 ± 0.030
Random forest	reg.	0.925 ± 0.019
Gradient boosting	reg.	0.934 ± 0.020
<i>Binned sequence</i>		
MLP, window means	rank	0.767 ± 0.039
Ridge, flattened	reg.	0.768 ± 0.030
GBM, window means	reg.	0.803 ± 0.059
MLP, summary stats	rank	0.821 ± 0.035
GBM, summary stats	reg.	0.842 ± 0.029
GBM, flattened	reg.	0.845 ± 0.020
MLP, flattened	rank	0.847 ± 0.032
Transformer	reg.	0.873 ± 0.026
Transformer	rank	0.873 ± 0.037
<i>Vector and binned sequence</i>		
Transformer	reg.	0.919 ± 0.021
Transformer	rank	0.931 ± 0.030
<i>Frame trace</i>		
Transformer†	reg.	0.888 ± 0.039
Transformer†	rank	0.897 ± 0.028

D. Results

Table III groups models by what they read. Every learned model beats every heuristic, so passive evidence past signal strength does carry something a client can use.

Gradient boosting on the hand-built vector is the most accurate at 0.934, with random forests at 0.925 behind it and the best heuristic at 0.867. Paired by partition, gradient boosting beats the random forest on all five, by 0.000 to 0.020, so the ordering at the top is consistent even though the two standard deviations overlap.

The binned sequence on its own supports less. Its best tabular encoding reaches 0.847 and the transformer 0.873, both short of every model on the vector. Give the same transformer the vector as well and it reaches 0.931, which is where gradient boosting already sits with no sequence at all. The trajectory

adds nothing measurable to the long-run averages. Dropping aggregation altogether does not help either: the frame-level transformer reaches 0.897 on the three partitions it was run on.

VI. INTERPRETATION AND NEXT STEPS

The results establish that passive evidence beyond signal strength is useful for AP selection. They do not establish that the order of that evidence matters. Gradient boosting on the feature vector reaches 0.934, against 0.867 for RSSI minus busy and 0.647 for strongest RSSI (Table III). Adding the binned sequence to the same vector moves a transformer to 0.931, so whatever the trajectory carries, the long-run averages already carry it.

The simulator gives temporal modeling little to exploit. After background traffic begins, the ratio of within-series to between-series standard deviation is 0.09 for frame rate, 0.09 for channel-busy fraction, and 0.17 for the option's own activity. Most measured variation lies between APs rather than within an AP over the window. An AP looks much the same at one second as at five. This is evidence that averaging loses little rather than proof that order is irrelevant, since a stationary-looking sequence can still carry autocorrelation, drift, or short-lived events.

The intraclass correlation of 0.988 is not evidence for either representation. It shows that throughput is highly reproducible once topology and target AP are fixed across radio seeds, not whether that throughput is best inferred from a time average, an ordered sequence, or something else.

The gap between the two transformer configurations is worth reading carefully. Given the sequence alone the model reaches 0.873, well behind the vector models; given both it reaches 0.931. Almost all of that comes from the averages, not from the bins. One reason the sequence alone struggles is sparsity: under a rotating radio roughly three-quarters of an option's bins carry no frames from it, because the radio was listening elsewhere. The vector is computed over the whole window and does not have that problem.

The frame-level experiment points the same way. On the three splits both were evaluated on under the ranking objective, the frame transformer reaches 0.897 against the binned transformer's 0.883, a gap smaller than the spread either shows across splits. Keeping individual frames buys nothing clear, though only under traffic as steady as this.

The simulator supports the premise of the project: a client can infer useful information about AP quality from passive pre-association observation, and learned combinations of that evidence beat signal strength alone by a wide margin. What it cannot show is whether order would matter in a network that moves, since this one barely does. The suggestions below are aimed at that.

A. Suggested Extensions

Two changes bear on the question this work leaves open. The first is to make the medium non-stationary on a timescale shorter than the observation window, and visibly so in a passive

trace: load that arrives and departs, stations that join and leave, interference that appears part-way through. The second is to shorten the label. A twelve-second mean of an already-stationary process is predicted best by the window mean, whereas a label measured over the second or two after the decision gives a trend something to extrapolate toward. Together they would make the value of temporal order identifiable rather than assumed.

The observation itself could be made more representative. The 802.11 BSS Load element, which advertises a station count and channel utilisation and which a client reads from a single beacon, is absent here, and it is the strongest baseline a real passive client has. The window could also be defined by the pre-association period rather than by complete silence, admitting the probe requests most clients actually send.

Two aspects of the scenario generator would repay attention. Channels are assigned by AP index over a fixed layout, so co-channel separation has no variance and reuse is tied to deployment size. The effect is a synthetic shortcut rather than leakage in the usual sense, since the number of discovered options is legitimately observable: option count determines AP count in 91.8% of choice sets, and AP count fixes the channel plan exactly, so a model can read the reuse regime off the size of its input rather than off radio evidence. The guard of Section I checks only that the observation is independent of which AP is joined, so it does not catch this; sampling the channel assignment would. Separately, the 30 m spacing pairs an obstructed-indoor path-loss exponent with an open-field geometry.

B. Ranking Versus Regression

The two objectives are not equally testable outside a simulator, and the difference favors the one this work treats as secondary.

Ranking asks which AP is best. Establishing ground truth for that answer requires joining each option under identical conditions, which is exactly what the matched replay of Section III provides and what no field measurement can. A ranking model's output is therefore not easily verifiable per decision in the field. Regression on the other hand can be tested reliably for accurate predictions of throughput.

This work reports both objectives because the simulator supports both. Work aimed at deployment should treat throughput prediction as the primary target, and derive the choice from it, rather than optimizing the choice directly and inheriting a claim that cannot be checked.

REFERENCES

- [1] P. Dely, A. Kessler, L. Chow, N. Bambos, N. Bayer, H. Einsiedler, and C. Peylo, "BEST-AP: Non-intrusive estimation of available bandwidth and its application for dynamic access point selection," *Computer Communications*, vol. 39, pp. 78–91, 2014, doi: 10.1016/j.comcom.2013.11.006.
- [2] T. Adame, M. Carrascosa, B. Bellalta, I. Pretel, and I. Etxebarria, "Channel load aware AP/Extender selection in home WiFi networks using IEEE 802.11k/v," *IEEE Access*, vol. 9, pp. 30095–30112, 2021, doi: 10.1109/ACCESS.2021.3059473.
- [3] W. Wong and S.-H. G. Chan, "Distributed joint AP grouping and user association for MU-MIMO networks," in *Proc. IEEE INFOCOM*, 2018, pp. 252–260, doi: 10.1109/INFOCOM.2018.8486284.

- [4] Y. Zhang and L. Dai, "Joint optimization of placement and coverage of access points for IEEE 802.11 networks," in *Proc. IEEE ICC*, 2020, pp. 1–7, doi: 10.1109/ICC40277.2020.9149098.
- [5] ns-3 Project, *ns-3 Model Library, Release 3.45*, 2025. [Online]. Available: <https://www.nsnam.org/docs/release/3.45/models/>

APPENDIX A HAND-BUILT FEATURES

The 35 columns available to the tabular models, derived from frames the candidate decoded before transmitting or associating and, for the CCA column, from its scanner's carrier-sense state. Every rate and fraction is normalized by time actually spent listening to the relevant channel, not by the observation window.

TABLE IV
THE FEATURE COLUMNS. AP-LEVEL AND CHANNEL-LEVEL FEATURES DESCRIBE THE OPTION; RELATIVE FEATURES DESCRIBE IT AGAINST ITS RIVALS, AND EXIST TO RESTORE COMPARISONS THAT PER-AP FILTERING REMOVED.

Column	Meaning
AP-level	
ap_rssi_mean	Mean beacon RSSI (dBm)
ap_rssi_std	Spread of beacon RSSI
ap_rssi_max	Strongest beacon seen
ap_rssi_min	Weakest beacon seen
ap_rssi_last	Most recent beacon RSSI
ap_beacons	Beacons decoded
ap_beacon_gap_mean	Mean interval between decoded beacons; set by scan revisit, not beacon period
ap_beacon_gap_std	Spread of that interval
ap_frames	Frames attributed to this BSS
ap_frames_per_s	The same, per second listening
ap_bytes_per_s	Bytes per second attributed to this BSS
ap_airtime_frac	Fraction of listening time this BSS occupied
ap_n_clients	Distinct transmitters other than the AP
ap_data_frames	Data frames from this BSS
ap_mean_data_rate	Mean PHY rate of its data frames
ap_retry_frac	Fraction of its frames marked retry
Channel-level	
chan_busy_frac	Channel occupancy from decoded frame airtime
chan_cca_busy_frac	Channel occupancy from PHY carrier sense; counts energy never decoded
chan_frames_per_s	All frames per second on this channel
chan_bytes_per_s	All bytes per second
chan_data_frac	Share of frames that are data
chan_mgmt_frac	Share that are management
chan_ctrl_frac	Share that are control
chan_retry_frac	Share marked retry
chan_mean_data_rate	Mean PHY rate of data frames
chan_n_bssids	Distinct BSSIDs heard
chan_n_tas	Distinct transmitters heard
chan_mean_rssi	Mean RSSI over all frames
Relative	
rel_n_options	Options in this choice set
rel_rssi_rank	Options strictly stronger than this one
rel_is_strongest	Whether no option is stronger
rel_rssi_margin_best_other	RSSI minus the best alternative's
rel_rssi_minus_mean	RSSI minus the option-set mean
rel_airtime_share	Share of observed airtime; denominator spans every BSS heard, so it does not sum to one
rel_clients_share	Share of observed transmitters